

Branch and bound

```
#include <iostream>

#include <vector>

using namespace std;

int solution_count = 0;

// ----- Print Board -----
void print_board(vector<vector<int>> &board, int n) {

    cout << endl;
    for (int i = 0; i < n; i++) {
        // Top Border
        for (int j = 0; j < n; j++)
            cout << "+---";
        cout << "+" << endl;
        // Row Content
        for (int j = 0; j < n; j++) {
            if (board[i][j] == 1)
                cout << "| Q ";
            else
                cout << "|  ";
        }
        cout << "|" << endl;
    }

    // Bottom Border
    for (int j = 0; j < n; j++)
        cout << "+---";
```

```

        cout << "+" << endl;
    }

// ----- Solve Using Branch & Bound -----

void solve(int row,
           vector<vector<int>> &board, vector<bool> &col, vector<bool> &diag1, vector<bool> &diag2,
           int n) {

    // All queens placed
    if (row == n) {
        solution_count++;
        cout << "\n===== Solution "
             << solution_count
             << " =====\n";
        print_board(board, n);
        return;
    }

    for (int c = 0; c < n; c++) {
        // Safety Check
        if (!col[c] && !diag1[row - c + n - 1] && !diag2[row + c]) {

            // ----- Place Queen -----
            cout << "\nPlacing Queen at (" << row << ", " << c << ")" << endl;
            board[row][c] = 1;
            col[c] = true;
            diag1[row - c + n - 1] = true;
            diag2[row + c] = true;

            // Recursive Call
            solve(row + 1, board, col, diag1, diag2, n);
        }
    }
}

```

```

// ----- Backtrack -----
cout << "Removing Queen from (" << row << ", " << c << ")" << endl;
board[row][c] = 0;
col[c] = false;
diag1[row - c + n - 1] = false;
diag2[row + c] = false;
}
}
}

```

```

// ----- Main Solver -----

```

```

void solve_n_queens_bb(int n) {
    vector<vector<int>> board(n, vector<int>(n, 0));
    vector<bool> col(n, false);
    vector<bool> diag1(2 * n - 1, false);
    vector<bool> diag2(2 * n - 1, false);

    solve(0, board, col, diag1, diag2, n);
}

```

```

// ----- Total Solutions -----

```

```

if (solution_count == 0) {
    cout << "\nNo solution exists\n";
}

```

```

else {
    cout << "\nTotal Solutions = " << solution_count << endl;
}
}

```

```

// ----- MAIN -----

```

```

int main() {

    int n;

    cout << "Enter number of queens: ";

    cin >> n;

    solve_n_queens_bb(n);

    return 0;

}

```

```

// #include <iostream>

// #include <vector>

// using namespace std;

// // Print board

// void print_board(vector<vector<int>> &board, int n) {

//     cout << endl;

//     for (int i = 0; i < n; i++) {

//         // Top border

//         for (int j = 0; j < n; j++)

//             cout << "+---";

//         cout << "+" << endl;

//         // Row content

//         for (int j = 0; j < n; j++) {

//             if (board[i][j] == 1)

//                 cout << "| Q ";

//             else

//                 cout << "|  ";

//         }

//         cout << "|" << endl;

//     }

//     // Bottom border

```

```

// for (int j = 0; j < n; j++)
//     cout << "+---";
// cout << "+" << endl;
// }

// // Solve using Branch & Bound
// bool solve(int row,
//     vector<vector<int>> &board,
//     vector<bool> &col,
//     vector<bool> &diag1,
//     vector<bool> &diag2,
//     int n) {
//     if (row == n) {
//         print_board(board, n);
//         return true; // stop after first solution
//     }
//     for (int c = 0; c < n; c++) {
//         // Check safety using bounding arrays
//         if (!col[c] &&
//             !diag1[row - c + n - 1] &&
//             !diag2[row + c]) {
//             // Place queen
//             board[row][c] = 1;
//             col[c] = true;
//             diag1[row - c + n - 1] = true;
//             diag2[row + c] = true;
//             if (solve(row + 1, board, col, diag1, diag2, n))
//                 return true;
//             // Backtrack
//             board[row][c] = 0;
//             col[c] = false;
//             diag1[row - c + n - 1] = false;

```

```

//      diag2[row + c] = false;
//  }
//  }
//  return false;
//}

// // Main function
// void solve_n_queens_bb(int n) {
//     vector<vector<int>> board(n, vector<int>(n, 0));
//     vector<bool> col(n, false);
//     vector<bool> diag1(2*n - 1, false);
//     vector<bool> diag2(2*n - 1, false);
//     if (!solve(0, board, col, diag1, diag2, n)) {
//         cout << "No solution exists" << endl;
//     }
// }

// int main() {
//     int n;
//     cout << "Enter number of queens: ";
//     cin >> n;
//     solve_n_queens_bb(n);
//     return 0;
// }

```

Backtracking

```
#include <iostream>

#include <vector>

#include <cmath>

using namespace std;

// ----- Global Solution Counter -----

int totalSolutions = 0;

// ----- Check if Safe -----

bool is_safe(vector<int> &board,int row,int col,int n) {

    for (int i = 0; i < row; i++) {

        // Same Column

        if (board[i] == col)

            return false;

        // Diagonal Check

        if (abs(board[i] - col) == abs(i - row))

            return false;

    }

    return true;

}

// ----- Print Board -----

void print_board(vector<int> &board,int n) {

    cout << endl;

    for (int i = 0; i < n; i++) {

        // Top Border

        for (int j = 0; j < n; j++)

            cout << "+---";

        cout << "+" << endl;

    }

}
```

```

// Row Content
for (int j = 0; j < n; j++) {
    if (board[i] == j)
        cout << " | Q ";
    else
        cout << " |  ";
}
cout << " | " << endl;
}

// Bottom Border
for (int j = 0; j < n; j++)
    cout << "+---";
cout << "+" << endl;
}

// ----- Solve Using Backtracking -----
void solve(vector<int> &board,int row,int n) {

// All Queens Placed
if (row == n) {
    totalSolutions++;

    cout << "\n===== Solution " << totalSolutions << " =====" << endl;
    print_board(board, n);
    return;
}

for (int col = 0; col < n; col++) {
    if (is_safe(board,row,col,n)) {

// ----- Place Queen -----

```



```

        cout << "\nPlacing Queen at ("<< row<< ", "<< col<< ")" << endl;

        board[row] = col;

        // Recursive Call
        solve(board, row + 1, n);

        // ----- Backtracking -----
        cout << "Removing Queen from ("<< row<< ", "<< col<< ")" << endl;
        board[row] = -1;
    }
}

// ----- Main Function -----
void solve_n_queens(int n) {
    vector<int> board(n, -1);
    solve(board, 0, n);

    // ----- Total Solutions -----
    if (totalSolutions == 0) {
        cout << "\nNo solution exists" << endl;
    }

    else {
        cout << "\nTotal Solutions = "<< totalSolutions<< endl;
    }
}

// ----- MAIN -----
int main() {
    int n;
    cout << "Enter number of queens: ";

```

```
    cin >> n;

    solve_n_queens(n);

    return 0;
}
```

```
// #include <iostream>

// #include <vector>

// #include <cmath>

// using namespace std;


// // Check if safe to place queen
// bool is_safe(vector<int> &board, int row, int col, int n) {
//     for (int i = 0; i < row; i++) {
//         // Same column
//         if (board[i] == col)
//             return false;
//         // Diagonal check
//         if (abs(board[i] - col) == abs(i - row))
//             return false;
//     }
//     return true;
// }

// // Print board
// void print_board(vector<int> &board, int n) {
//     cout << endl;
//     for (int i = 0; i < n; i++) {
//         // Horizontal line
//         for (int j = 0; j < n; j++)
//             cout << "+---";
```

```

//    cout << "+" << endl;
//    // Row with queen
//    for (int j = 0; j < n; j++) {
//        if (board[i] == j)
//            cout << "| Q ";
//        else
//            cout << "|  ";
//    }
//    cout << "|" << endl;
// }
// // Bottom line
// for (int j = 0; j < n; j++)
//     cout << "+---";
// cout << "+" << endl;
//}

// // Solve using backtracking
// bool solve(vector<int> &board, int row, int n) {
//     if (row == n) {
//         print_board(board, n);
//         return true; // stop after first solution
//     }
//     for (int col = 0; col < n; col++) {
//         if (is_safe(board, row, col, n)) {
//             board[row] = col;
//             if (solve(board, row + 1, n))
//                 return true;
//             // Backtrack
//             board[row] = -1;
//         }
//     }
//     return false;

```

```
// }
```

```
// // Main function
```

```
// void solve_n_queens(int n) {
```

```
//     vector<int> board(n, -1);
```

```
//     if (!solve(board, 0, n)) {
```

```
//         cout << "No solution exists" << endl;
```

```
//     }
```

```
// }
```

```
// int main() {
```

```
//     int n;
```

```
//     cout << "Enter number of queens: ";
```

```
//     cin >> n;
```

```
//     solve_n_queens(n);
```

```
//     return 0;
```

```
// }
```